APPENDIX H

Estimation theory

There are many ways to take measurements from spacecraft sensors and use them to produce and estimate of the spacecraft's orientation. Kalman filters, while not the only approach, are often a good approach to this problem.

There are several types of attitude estimation algorithms that are based on the Kalman filter. These include (in order of complexity),

1. The Kalman filter;
2. The extended Kalman filter;
3. The unscented Kalman filter.

A Kalman filter consists of a measurement model that relates measurements to the spacecraft state, and a dynamical model of the time evolution of the state. The three types of Kalman filters listed above approach the problem in different ways.

The first method estimates spacecraft attitude using (usually approximate) static, linear relationships between the attitude and sensor measurements. The dynamical model is also linear.

The other two methods can utilize the nonlinear measurement and dynamical models directly. They are much more complex and require more flight software for their implementation.

This chapter will start with estimation system and then move on to Kalman filters.

H.1. Estimation theory

Given a physical system that can be modeled in the form

$$\dot{x} = f(x, u, t) \tag{H.1}$$

the goal is to get the best possible estimate of x. Take, for example, a very simple physical system in continuous time

$$\dot{x} = -ax + u \tag{H.2}$$

$$y = x \tag{H.3}$$

with only one parameter, a, and equations that are first order and linear. The input to the system is u and the system state is x. The sensor measurement y is of the system state, i.e., $y = x$.

A simple estimator algorithm consists of numerically integrating the following dynamical equation:

$$\dot{x}_E = ax_E + u_E + L(y_M - y_E) \tag{H.4}$$

In the above equation subscript E emphasizes that we are integrating estimates of the corresponding variables, and y_M is the actual sensor measurement. We note that, as per Eq. (H.3), $y_E = x_E$, i.e., the estimated (or expected) sensor measurement is the same as the estimated state of the system in this example. L is called the gain of the estimator. Effectively, L finds the difference between the estimated measurement and the actual measurement. It produces an input that is added to u_E that pushes the estimate, x_E towards x.

The transfer function between the estimated x and the measurement y_M and estimated input u_E can be computed by taking Laplace transforms of both sides of Eq. (H.4):

$$x_E(s) = \frac{Ly_M}{s + (a - L)} + \frac{u_E}{s + (a - L)} \tag{H.5}$$

Thus we have a first-order, low-pass filter with a DC gain of $L/(a - L)$ from y_M to x_E and a first-order, low-pass filter with a DC gain of $1/(a_L)$ from u_E to x_E. We note that if $L = 0$ the estimator (Eq. (H.4)) just propagates the estimated inputs. On the other hand, if L is large the estimator ignores the inputs and effectively only uses the measurements. The filter is less responsive to high-frequency components of the input measurement signal – the "cut-off frequency" of the filter is $a - L$.

If the bandwidth of u is known, L can be selected so that frequencies beyond those contained in u would be attenuated. Given a knowledge of the measurement noise and the uncertainty in the plant one can optimally pick L using quadratic estimator theory. The optimal gain is found from the equations

$$\dot{p} = -2ap + q - \frac{p^2}{r} \tag{H.6}$$

$$L = \frac{p}{r} \tag{H.7}$$

$$E[v(t)v^T(t')] = rb(t - t') \tag{H.8}$$

$$E[w(t)w^T(t')] = qb(t - t') \tag{H.9}$$

where $E[.]$ is expectation. Given the model

$$\dot{x} = -ax + u + w \tag{H.10}$$

$$y = x + v \tag{H.11}$$

where w and v are white-noise processes, d has units of seconds, q represents uncertainty in the plant model and in the inputs u, and r represents measurement noise.

In steady state (as t approaches ∞)

$$L = \sqrt{a^2 + q/r} - a \tag{H.12}$$

In the limit

$$L = \begin{cases} 0 & r/q \to \infty \\ \sqrt{q/r} & q/r \to \infty \end{cases} \tag{H.13}$$

This result means that for very noisy signals, or perfect knowledge of the plant, L should be set to zero. This can also be implemented as a time-varying filter. The time-varying gain is

$$L(t) = \beta - a + \frac{2\beta}{\left(\frac{p(0)+r(\beta+a)}{p(0)-r(\beta-a)}\right) e^{2\beta t} - 1} \tag{H.14}$$

$$\beta = \sqrt{a^2 + q/r} \tag{H.15}$$

$L(0) = \frac{p(0)}{r}$, which means that the bigger the uncertainty in x initially, the bigger the gain. This gain is independent of the measurements or the state. The steady-state value is independent of $p(0)$. Therefore given a, q, and r, it will always converge to the same value. This will be true about all time-varying filters derived this way including the extended Kalman filters.

The gain is dependent on the ratio between the plant and measurement uncertainty and the plant parameters. If a time-varying filter is used, the gain will always vary in the same way, given the same initial covariance. The white-noise approximation for the plant noise is usually not very good. It is often necessary to augment the state equations with additional equations modeling the plant and disturbance uncertainty. The ultimate form of the estimator is a first-order, low-pass filter, except that an estimate of the disturbance is added. If this is unavailable the estimator is a simple first-order, low-pass filter.

Usually, data for gyros is provided as spectral densities. Thus these equations can be used directly to determine the covariances of the estimates in steady-state. They provide a useful check for the discrete-time Kalman-filter equations.

The matrix Riccati equation is

$$\dot{P} = FP + PF^T - PH^T R^{-1} HP + Q \tag{H.16}$$

which relates the state covariance, P, to the plant model, F, the measurement matrix, H, the measurement noise matrix, H, and the plant noise spectral density matrix, Q.

In this case, F is the state matrix

$$F = \begin{bmatrix} 0 & 1 \\ 0 & 0 \end{bmatrix} \tag{H.17}$$

and H is the measurement equation

$$H = \begin{bmatrix} 1 & 0 \end{bmatrix} \tag{H.18}$$

Q is the spectral density matrix for the plant and R is the spectral density matrix for the measurement. Expanding the covariance equation with the derivative set to zero gives

$$\begin{bmatrix} p_{12} & p_{22} \\ 0 & 0 \end{bmatrix} + \begin{bmatrix} p_{12} & 0 \\ p_{22} & 0 \end{bmatrix} - \frac{1}{r}\begin{bmatrix} p_{11}^2 & p_{11}p_{12} \\ p_{11}p_{12} & p_{12}^2 \end{bmatrix} + \begin{bmatrix} q_{11} & 0 \\ 0 & q_{22} \end{bmatrix} = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix} \tag{H.19}$$

Writing out the scalar equations gives

$$2p_{12} + q_{11} - \frac{p_{11}^2}{r} = 0 \tag{H.20}$$

$$p_{22} = \frac{p_{11}p_{12}}{r} \tag{H.21}$$

$$p_{12} = \sqrt{rq_{22}} \tag{H.22}$$

Solving for elements of P,

$$p_{11} = \sqrt{r(q_{11} + 2\sqrt{q_{22}})} \tag{H.23}$$

$$p_{22} = \sqrt{\frac{q_{22}(q_{11} + 2\sqrt{q_{22}})}{r}} \tag{H.24}$$

$$p_{12} = \sqrt{rq_{22}} \tag{H.25}$$

If there were no plant noise, i.e., if $q_{11} = q_{22} = 0$ the steady-state covariance matrix would go to zero, as would be expected.

H.1.1 Conversion from continuous to discrete time

The continuous-time data assumes continuous measurements. In practice, the sensors are sampled. Converting the continuous spectral densities to covariances is done by

$$Q_k = \tau Q \tag{H.26}$$

$$R_k = \frac{R}{\tau} \tag{H.27}$$

where t is the sampling time. In this case, the units of Q_k and R_k are rad^2 and (rad/s)^2.

H.2. The Kalman-filter algorithm

Kalman filters are recursive estimators that adjust their gains based on a model for the system. Essentially, that automates the process of finding the passband for a system. The simplest Kalman filter is for a continuous linear dynamical model, known as the plant, of the form

$$\dot{x} = ax + bu \tag{H.28}$$

$$y = cx \tag{H.29}$$

where a and b are transformed to discrete time given the desired time step. The time step needs to be short enough so that the frequency

$$\frac{2\pi}{\Delta t} \tag{H.30}$$

is higher than the dynamics of a. Discretizing can be done in many ways. In the book, we use the zero-order hold (ZOH). This forms the basis of the discrete-time linear Kalman filter,

$$x_{k+1} = \Phi x_k + \Gamma u_k + \eta_x \tag{H.31}$$

$$y_k = Hx_k + \eta_y \tag{H.32}$$

where x is the state, Φ is the state-transition matrix, Γ is the input matrix, y_k is the measurement, and H is the measurement matrix. H and c are the same. Assume that there is a Gaussian white-noise input to the plant and Gaussian white noise on the measurement. The plant white noise is due to unmodeled inputs and modeling errors. Modeling errors are due to several reasons

1. A reduced-order model is used;
2. There is uncertainty in the values for Φ, Γ, and H matrices;
3. The system is nonlinear and the linear matrices are only valid at the operating point;
4. There are poorly known inputs to the system.

The Kalman filter has two parts. One is an update due to the measurement. The second is due to the prediction of change in the state, x, due to the dynamics. We leave out the k indices,

$$K = \frac{pH^T}{HPH^T + R} \tag{H.33}$$

$$x_e = x_e + K(y - Hx_e) \tag{H.34}$$

$$P = (E - KH)P \tag{H.35}$$

where E is the identity matrix, Q is the covariance matrix for the model uncertainty, P is the state covariance matrix, and R is the measurement covariance matrix. x_e is the

estimated state and y is the measurement.

$$x_e = \Phi x_e + \Gamma u \tag{H.36}$$

$$P = \Phi P \Phi^T + \Gamma Q \Gamma^T \tag{H.37}$$

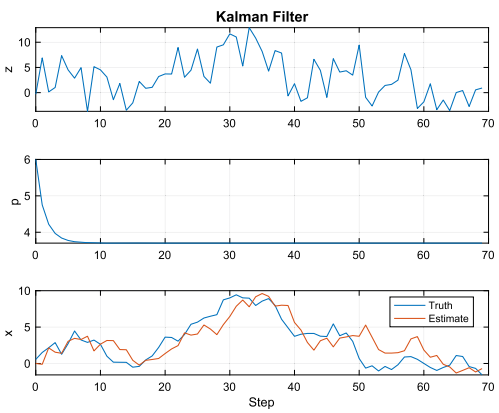
The code in SimpleKalmanFilter is for the system

$$x_{k+1} = x_k + u_k \tag{H.38}$$

$$y_k = x_k \tag{H.39}$$

where x is a scalar. This a single integrator. The results, which are dependent on the covariances, are shown in Example H.1. The estimate tracks the state reasonably well. The covariance reaches a predetermined value based on the ratio of Q and R. The covariance is like clockwork, it is independent of the measurements.

```
n     = 70; % Number of steps
p0    = 6; % Initial covariance
r     = 10; % Measurement noise
q     = 1; % Plant noise
h     = 1; % Measurement matrix
phi   = 1; % State transition matrix
gamma = 1; % Input matrix
x     = 1; % Initial state
xE    = 0; % Estimated state
u     = 0; % Input
p     = p0;

xP    = zeros(4,n);
for j = 1:n

        % Truth model
  z = h*x + sqrt(r)*randn; % Measurement
  x = phi*x + gamma*u + sqrt(q)*randn;

  % Store variables to plot
  xP(:,j) = [z;p;x;xE];

  % Kalman Filter

  % Update
  k  = p*h'/(h*p*h' + r);
  xE = xE + k*(z - h*xE);
  p  = (1 - k*h)*p; % Measurement update of
       covariance

  % Prediction
  xE = phi*xE + gamma*u;
  p  = phi*p*phi' + gamma*q*gamma';

end

yL = {'z' 'p' 'x'};
leg = {{},{},{'Truth','Estimate'}};

Plot2D(0:n-1,xP,'Step',yL,'Kalman Filter',...
       'lin',{'1' '2' '[3 4]'},[],[],[],[], ...
       leg);
```



Example H.1: Single-integrator Kalman-filter results.

H.3. Bayesian derivation

Kalman filters were originally derived using least-squares. Kalman filters can also be derived from the Bayes theorem. What is Bayes theorem? Bayes theorem is

$$P(A_i|B) = \frac{P(B|A_i)P(A_i)}{\sum P(B|A_i)} \tag{H.40}$$

$$P(A_i|B) = \frac{P(B|A_i)P(A_i)}{P(B)} \tag{H.41}$$

which is just the probability of A_i given B. P means "probability". The vertical bar | means "given". This assumes that the probability of B is not zero, that is $P(B) \neq 0$. In the Bayesian interpretation, the theorem introduces the effect of evidence on belief. This provides a rigorous framework for incorporating any data for which there is a degree of uncertainty. Put simply, given all the evidence (or data) to date, Bayes theorem allows you to determine how new evidence affects the belief. In the case of state estimation, this is the belief in the accuracy of the state estimate.

Fig. H.1 shows the Kalman-filter family and how it relates to the Bayesian filter. In this book, we are covering only the ones in the colored boxes. The complete derivation of the Kalman filter is given below; this provides a coherent framework for all Kalman-filtering implementations. The different filters fall out of the Bayesian models based on assumptions about the model and sensor noise and the linearity or nonlinearity of the measurement and dynamics models. Let us look at the branch that is colored blue. Addi-

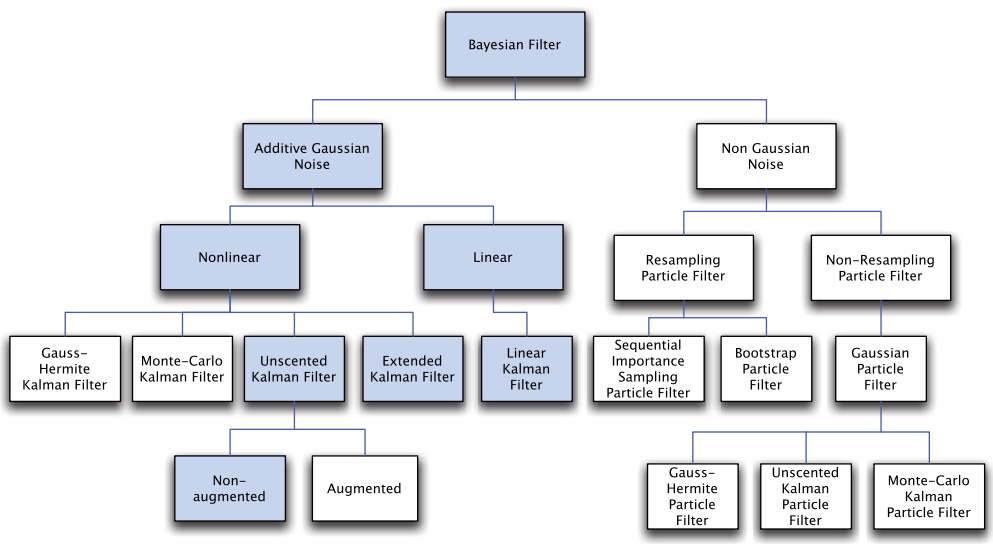


Figure H.1 The Kalman-filter family tree. All are derived from a Bayesian filter. This book covers those in colored boxes.

tive Gaussian noise filters can be linear or nonlinear depending on the type of dynamical and measurement models. In many cases, you can take a nonlinear system and linearize it about the normal operating conditions. You can then use a linear Kalman filter. For example, a spacecraft dynamical model is nonlinear and an Earth sensor that measures the Earth's chordwidth for roll and pitch information is nonlinear. However, if we are only concerned with Earth pointing, and small deviations from nominal pointing, we can linearize both the dynamical equations and measurement equations and use a linear Kalman filter.

If nonlinearities are important we have to use a nonlinear filter. The extended Kalman filter uses partial derivatives of the measurement and dynamical equations. These are computed each time step or with each measurement input. In effect, we are linearizing the system each step and using the linear equations. We do not have to do a linear state propagation, that is propagating the dynamical equations, and could propagate them using numerical integration. If we can get analytical derivatives of the measurement and dynamical equations this is a reasonable approach. If there are singularities in any of the equations this may not work.

The unscented Kalman filter uses the nonlinear equations directly. There are two forms, augmented and nonaugmented. In the former, we created an augmented state vector that includes both the states and the state and measurement noise variables. This may result in better results at the expense of more computation.

All of the filters in this section are Markov, that is the current dynamical state is entirely determined from the previous state. Particle filters are not addressed in this book. They are a class of Monte Carlo methods. Monte Carlo (named after the famous casino) methods are computational algorithms that rely on random sampling to obtain results. For example, a Monte Carlo approach to our oscillator simulation would be to use the MATLAB® function nrandn to generate the accelerations. We would run many tests to verify that our mass moves as expected.

Our derivation will use the notation $\mathrm{N}(\mu, \sigma^2)$ to represent a normal variable. A normal variable is another word for a Gaussian variable. Gaussian means it is distributed as the normal distribution with mean μ (average) and variance σ^2. The following code from Gaussian computes a Gaussian or normal distribution around a mean of 2 for a range of standard deviations. Example H.2 shows a plot. The height of the plot indicates how likely a given measurement of the variable is to have that value. Example H.2 generates a Gaussian distribution.

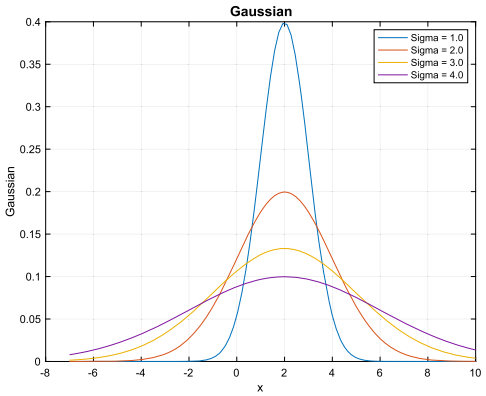
Given the probabilistic state-space model in discrete time [1]

$$x_k = f_k(x_{k-1}, w_{k-1}) \tag{H.42}$$

where x is the state vector and w is the noise vector, the measurement equation is

$$y_k = h_k(x_k, v_n) \tag{H.43}$$

```
mu              = 2;              % Mean
sigma           = [1 2 3 4]; % Standard deviation
n               = length(sigma);
x               = linspace(-7,10);

%% Simulation
xPlot = zeros(n,length(x));
s     = cell(1,n);

for k = 1:length(sigma)
  s{k}        = sprintf('Sigma = %3.1f',sigma(k));
  f           = -(x-mu).^2/(2*sigma(k)^2);
  xPlot(k,:) = exp(f)/sqrt(2*pi*sigma(k)^2);
end

%% Plot the results
Plot2D(x,xPlot,'x','Gaussian','Gaussian')
legend(s)
```



Example H.2: Gaussian distribution.

where v_n is the measurement noise. This has the form of a hidden Markov model (HMM) because the state is hidden.

If the process is Markovian, then the future state x_k is dependent only on the current state x_{k-1} and is not dependent on the past states. This can be expressed in the equation,

$$p(x_k|x_{1:k-1}, y_{1:k-1}) = p(x_k|x_{k-1}) \tag{H.44}$$

The | means given. In this case, the first term is read as "the probability of x_k given $x_{1:k-1}$ and $y_{1:k-1}$". This is the probability of the current state given all past states and all measurements up to the $k-1$ measurement. The past, x_{k-1} is independent of the future given the present,

$$p(x_{k-1}|x_{k:T}, y_{k:T}) = p(x_{k-1}|x_k) \tag{H.45}$$

where T is the last sample and the measurements y_k are conditionally independent given x_k; that is, they can be determined using only x_k and are not dependent on $x_{1:k}$ or $y_{1:k-1}$. This can be expressed as

$$p(y_k|x_{1:k}, y_{1:k-1}) = p(y_k|x_k) \tag{H.46}$$

We can define the recursive Bayesian optimal filter that computes the distribution

$$p(x_k|y_{1:k}) \tag{H.47}$$

given:

- the prior distribution $p(x_0)$, where x_0 is the state prior to the first measurement;
- the state-space model

$$x_k \sim p(x_k|x_{k-1}) \tag{H.48}$$

$$y_k \sim p(y_k|x_k) \tag{H.49}$$

- the measurement sequence $y_{1:k} = y_1, \ldots, y_k$.
  Computation is based on the recursion rule

$$p(x_{k-1}|y_{1:k-1}) \rightarrow p(x_k|y_{1:k}) \tag{H.50}$$

This means we get the current state x_k from the prior state x_{k-1} and all the past measurements $y_{1:k-1}$. Assume we know the posterior distribution of the previous time step

$$p(x_{k-1}|y_{1:k-1}) \tag{H.51}$$

The joint distribution of x_k, x_{k-1} given $y_{1:k-1}$ can be computed as

$$p(x_k, x_{k-1}|y_{1:k-1}) = p(x_k|x_{k-1}, y_{1:k-1})p(x_{k-1}|y_{1:k-1}) \tag{H.52}$$

$$= p(x_k|x_{k-1})p(x_{k-1}|y_{1:k-1}) \tag{H.53}$$

because this is a Markov process. Integrating over x_{k-1} gives the prediction step of the optimal filter, which is the Chapman–Kolmogorov equation

$$p(x_k|y_{1:k-1}) = \int p(x_k|x_{k-1}, y_{1:k-1})p(x_{k-1}|y_{1:k-1})dx_{k-1} \tag{H.54}$$

The Chapman–Kolmogorov equation is an identity relating the joint probability distributions of different sets of coordinates on a stochastic process. The measurement update state is found from the Bayes rule

$$P(x_k|y_{1:k}) = \frac{1}{C_k}p(y_k|x_k)p(x_k|y_{k-1}) \tag{H.55}$$

$$C_k = p(y_k|y_{1:k-1}) = \int p(y_k|x_k)p(x_k|y_{1:k-1})dx_k \tag{H.56}$$

where C_k is the probability of the current measurement, given all past measurements.

If the noise is additive and Gaussian with the state covariance Q_n and the measurement covariance R_n, the model and measurement noise have zero mean, we can write the state equation as

$$x_k = f_k(x_{k-1}) + w_{k-1} \tag{H.57}$$

where x is the state vector and w is the noise vector. The measurement equation becomes

$$y_k = h_k(x_k) + v_n \tag{H.58}$$

Given that Q is not time dependent we can write

$$p(x_k|x_{k-1}, y_{1:k-1}) = \mathrm{N}(x_k; f(x_{k-1}), Q) \tag{H.59}$$

where we recall that N is a normal variable, in this case with mean $x_k; f(x_{k-1})$, which means (x_k given $f(x_{k-1})$ and variance Q. We can now write the prediction step Eq. (H.54) as

$$p(x_k|y_{1:k-1}) = \int \mathrm{N}(x_k; f(x_{k-1}), Q)p(x_{k-1}|y_{1:k-1})dx_{k-1} \tag{H.60}$$

We need to find the first two moments of x_k. A moment is the expected value (or mean) of the variable. The first moment is of the variable, the second is of the variable squared and so forth. They are

$$E[x_k] = \int x_k p(x_k|y_{1:k-1})dx_k \tag{H.61}$$

$$E[x_k x_k^T] = \int x_k x_k^T p(x_k|y_{1:k-1})dx_k \tag{H.62}$$

where E means expected value, $E[x_k]$ is the mean, and $E[x_k x_k^T]$ is the covariance. Expanding the first moment and using the identity $E[x] = \int x\mathrm{N}(x; f(s), \Sigma)dx = f(s)$, where s is any argument,

$$E[x_k] = \int x_k \left[\int \mathrm{d}(x_k; f(x_{k-1}), Q)p(x_{k-1}|y_{1:k-1})dx_{k-1} \right] dx_k \tag{H.63}$$

$$= \int x_k \left[\int \mathrm{N}(x_k; f(x_{k-1}), Q)dx_k \right] p(x_{k-1}|y_{1:k-1})dx_{k-1} \tag{H.64}$$

$$= \int f(x_{k-1})p(x_{k-1}|y_{1:k-1})dx_{k-1} \tag{H.65}$$

Assuming that $p(x_{k-1}|y_{1:k-1}) = \mathrm{N}(x_{k-1}; \hat{x}_{k-1|k-1}, P^{xx}_{k-1|k-1})$ where P^{xx} is the covariance of x and noting that $x_k = f_k(x_{k-1}) + w_{k-1}$ we get

$$\hat{x}_{k|k-1} = \int f(x_{k-1})\mathrm{N}(x_{k-1}; \hat{x}_{k-1|k-1}, P^{xx}_{k-1|k-1})dx_{k-1} \tag{H.66}$$

For the second moment

$$E[x_k x_k^T] = \int x_k x_k^T p(x_k|y_{1:k-1})dx_k \tag{H.67}$$

$$= \int \left[\int \mathrm{N}(x_k; f(x_{k-1}), Q)x_k x_k^T dx_k \right] p(x_{k-1}|y_{1:k-1})dx_{k-1} \tag{H.68}$$

which results in

$$P^{xx}_{k|k-1} = Q + \int f(x_{k-1})f^T(x_{k-1})\mathrm{N}(x_{k-1}; \hat{x}_{k-1|k-1}, P^{xx}_{k-1|k-1})dx_{k-1} - \hat{x}^T_{k|k-1}\hat{x}_{k|k-1} \tag{H.69}$$

The covariance for the initial state is Gaussian and is P^{xx}_0. The Kalman filter can be written without further approximations as

$$\hat{x}_{k|k} = \hat{x}_{k|k-1} + K_n\left[y_k - \hat{y}_{k|k-1}\right] \tag{H.70}$$

$$P^{xx}_{k|k} = P^{xx}_{k|k-1} - K_n P^{yy}_{k|k-1} K_n^T \tag{H.71}$$

$$K_n = P^{xy}_{k|k-1}\left[P^{yy}_{k|k-1}\right]^{-1} \tag{H.72}$$

where K_n is the Kalman gain and P^{yy} is the measurement covariance. The solution of these equations requires the solution of five integrals of the form

$$I = \int g(x)\mathrm{N}(x; \hat{x}, P^{xx})dx \tag{H.73}$$

The three integrals needed by the filter are:

$$P^{yy}_{k|k-1} = R + \int h(x_n)h^T(x_n)\mathrm{N}(x_n; \hat{x}_{k|k-1}, P^{xx}_{k|k-1})dx_k - \hat{x}^T_{k|k-1}\hat{y}_{k|k-1} \tag{H.74}$$

$$P^{xy}_{k|k-1} = \int x_n h^T(x_n)\mathrm{N}(x_n; \hat{x}_{k|k-1}, P^{xx}_{k|k-1})dx \tag{H.75}$$

$$\hat{y}_{k|k-1} = \int h(x_k)\mathrm{N}(x_k; \hat{x}_{k|k-1}, P^{xx}_{k|k-1})dx_k \tag{H.76}$$

Assume we have a model of the form

$$x_k = A_{k-1}x_{k-1} + B_{k-1}u_{k-1} + q_{k-1} \tag{H.77}$$

$$y_k = H_k x_k + r_k \tag{H.78}$$

where

- $x_k \in \Re^n$ is the state of system at time k.
- m_k is the mean state at time k.
- A_{k-1} is the state transition matrix at time $k-1$.
- B_{k-1} is the input matrix at time $k-1$.
- u_{k-1} is the input at time $k-1$.
- $q_{k-1}, \mathrm{N}(0, Q_k)$, is the Gaussian process noise at time $k-1$.
- $y_k \in \Re^m$ is the measurement at time k.
- H_k is the measurement matrix at time k. This is found from the Jacobian (derivatives) of $h(x)$.
- $r_k = \mathrm{N}(0, R_k)$ is the Gaussian measurement noise at time k.
- The prior distribution of the state is $x_0 = \mathrm{N}(m_0, P_0)$ where parameters m_0 and P_0 contain all prior knowledge about the system. m_0 is the mean at time zero and P_0 is the covariance. Since our state is Gaussian this completely describes the state.
- $\hat{x}_{k|k-1}$ is the mean of x at k given $\hat{x}$ at $k-1$.
- $\hat{y}_{k|k-1}$ is the mean of y at k given $\hat{x}$ at $k-1$.

$\Re^n$ means real numbers in a vector of order n, that is the state has n quantities. In probabilistic terms the model is

$$p(x_k|x_{k-1}) = \mathrm{N}(x_k; A_{k-1}x_{k-1}, Q_k) \tag{H.79}$$
$$p(y_k|x_k) = \mathrm{N}(y_k; H_k x_k, R_k) \tag{H.80}$$

The integrals become simple matrix equations. In the following equations P_k^- means the covariance prior to the measurement update.

$$P_{k|k-1}^{yy} = H_k P_k^- H_k^T + R_k \tag{H.81}$$
$$P_{k|k-1}^{xy} = P_k^- H_k^T \tag{H.82}$$
$$P_{k|k-1}^{xx} = A_{k-1} P_{k-1} A_{k-1}^T + Q_{k-1} \tag{H.83}$$
$$\hat{x}_{k|k-1} = m_k^- \tag{H.84}$$
$$\hat{y}_{k|k-1} = H_k m_k^- \tag{H.85}$$

The prediction step becomes

$$m_k^- = A_{k-1} m_{k-1} \tag{H.86}$$
$$P_k^- = A_{k-1} P_{k-1} A_{k-1}^T + Q_{k-1} \tag{H.87}$$

The first term in the above covariance equation propagates the covariance based on the state-transition matrix, A. Q_{k+1} adds to this to form the next covariance. Process noise Q_{k+1} is a measure of the accuracy of the mathematical model, A, in representing the system and includes the effects of unmodeled inputs. For example, suppose A was a mathematical model that damped all states to zero. Without Q, P would go to zero. However, if we were not that certain about the model, the covariance would never be less than Q. Picking Q can be difficult. In a dynamical system with uncertain disturbances, you can compute the standard deviation of the disturbances to compute Q. If the model, A, is uncertain then you might do a statistical analysis of the range of models. Or you can try different Q in simulation and see which ones work best.

The update step is

$$v_k = y_k - H_k m_k^- \tag{H.88}$$
$$S_k = H_k P_k^- H_k^T + R_k \tag{H.89}$$
$$K_k = P_k^- H_k^T S_k^{-1} \tag{H.90}$$
$$m_k = m_k^- + K_k v_k \tag{H.91}$$
$$P_k = P_k^- - K_k S_k K_k^T \tag{H.92}$$

where S_k is an intermediate quantity and v_k is the residual. The residual is the difference between the measurement and your estimate of the measurement given the estimated states. R is the covariance matrix of the measurements. If the noise is not white, a different filter should be used. White noise has equal energy at all frequencies. Many types of noise, such as the noise from an imager, are not white noise but are bandlimited,

that is it has noise in a limited range of frequencies. You can add additional states to A to model the noise better. For example, adding a low-pass filter to the band limits the noise. This makes the size of the A matrix larger requiring more computation.

H.4. Extended Kalman filter

The recursive least-squares method is implemented with an iterated extended Kalman filter measurement update. The state update is

$$\dot{x} = f(x, u, t) \tag{H.93}$$

The covariance update requires the Jacobian, that is the partial of $f(x, u, t)$.

$$A(x, u, t) = \frac{\partial f}{\partial x} \tag{H.94}$$

so that

$$f(x, u, t) \approx A(x, u, t)x \tag{H.95}$$

A is then discretized. This must be done each time step. The partial can only be done if there are no singularities or discontinuities in the right-hand side.

Iteration is applied to the measurement update. It reduces the measurement error due to having an inexact state estimate that would lead to an inexact measurement update. The equation is

$$\begin{aligned} x_{k,i+1} &= x_{k,0} + K_{k,i}\left[y - h(x_{k,i}) - H(x_{k,i})(x_{k,0} - x_{k,i})\right] \\ K_{k,i} &= P_{k,0}H^T(x_{k,i})\left[H(x_{k,i}P_{k,0}H^T(x_{k,i} + R_k\right]^{-1} \\ P_{k,i+1} &= \left[1 - K_{k,i}H(x_{k,i}\right]P_{k,0} \end{aligned} \tag{H.96}$$

where i denotes the iteration. Note that on the right-hand side P and R are always from step k and are not updated during the iteration.

H.5. Unscented Kalman filter

The UKF can achieve greater estimation performance than the extended Kalman filter (EKF) through the use of the unscented transformation (UT). The UT allows the UKF to capture first- and second-order terms of the nonlinear system [2]. The state-estimation algorithm employed is that given by van der Merwe [3] and the parameter-estimation algorithm given by [2]. Unlike the EKF, the UKF does not require any derivatives or Jacobians of either the state equations or measurement equations. Instead of just propagating the state, the filter propagates the state and additional sigma points that are the states plus the square roots of rows or columns of the covariance matrix.

Thus the state and the state plus a standard deviation are propagated. This captures the uncertainty in the state. It is not necessary to numerically integrate the covariance matrix. No specialized, linearized or simplified simulation is needed. Thus the algorithms can update the actual simulations.

The UKF has three parts:

1. Initialization;
2. UKF prediction;
3. UKF update.

The weights are computed once at the initialization of the filter [4]

$$W_m^0 = \frac{\lambda}{n+\lambda} \tag{H.97}$$

$$W_c^0 = \frac{\lambda}{n+\lambda} + 1 - \alpha^2 + \beta \tag{H.98}$$

$$W_m^i = W_i^c = \frac{1}{2(n+\lambda)} \tag{H.99}$$

$$W_c^i = W_m^i \tag{H.100}$$

where

$$\lambda = \alpha^2(n+\kappa) - n \tag{H.101}$$

$$\gamma = \sqrt{L+\lambda} \tag{H.102}$$

The constant α determines the spread of the sigma points around $\hat{X}$ and is usually set to between 10^{-4} and 1. β incorporates prior knowledge of the distribution of x and is 2 for a Gaussian distribution. κ is set to 0 for state estimation.

$$c = \alpha^2(n+\kappa) \tag{H.103}$$

$$w_m = \begin{bmatrix} W_m^0 & \cdots & W_m^{2n} \end{bmatrix}^T \tag{H.104}$$

$$W = \left(I - \begin{bmatrix} w_m & \cdots & w_m \end{bmatrix}\right) \times \begin{bmatrix} W_c^0 & 0 & 0 \\ 0 & \ddots & 0 \\ 0 & 0 & W_c^{2n} \end{bmatrix} \times \left(I - \begin{bmatrix} w_m & \cdots & w_m \end{bmatrix}\right)^T \tag{H.105}$$

where I is the identity matrix with dimensions $2n \times 2n$.

H.6. UKF state-prediction step

The prediction step is as follows

$$X_{k-1} = \begin{bmatrix} m_{k-1} & m_{k-1} & m_{k-1} \end{bmatrix} + \sqrt{c}\begin{bmatrix} 0 & \sqrt{P_{k-1}} & -\sqrt{P_{k-1}} \end{bmatrix} \tag{H.106}$$

$$\hat{X}_k = f(X_{k-1}, k-1) \tag{H.107}$$
$$m_k^- = \hat{X}_k w_m \tag{H.108}$$
$$P_k^- = \hat{X}_k W \hat{X}_k^T + Q_{k-1} \tag{H.109}$$

where m is the mean state and Q_{k-1} is the model covariance matrix. The square root of P can be found in numerous ways. One is the matrix square root. A more efficient method is to take the Cholesky decomposition of P, which is an approximation of the matrix square root.

The measurement update step is

$$X_{k-1}^- = \begin{bmatrix} m_{k-1}^- & m_{k-1}^- & m_{k-1}^- \end{bmatrix} + \sqrt{c}\begin{bmatrix} 0 & \sqrt{P_{k-1}^-} & -\sqrt{P_{k-1}^-} \end{bmatrix} \tag{H.110}$$
$$Y_k^- = h(X_{k-1}^-, k) \tag{H.111}$$
$$\mu_k = Y_k^- w_m \tag{H.112}$$
$$S_k = Y_k^- W \left[Y_k^-\right]^T + R_k \tag{H.113}$$
$$C_k = X_k^- W \left[Y_k^-\right]^T \tag{H.114}$$

where R_k is the measurement noise covariance matrix. Compute the gain K_k, mean state m_k, and covariance P_k,

$$K_k = C_k S_k^- 1 \tag{H.115}$$
$$m_k = m_k^- + K_k\left[y_k - \mu_k\right] \tag{H.116}$$
$$P_k = P_k^- - K_k S_k K_k^T \tag{H.117}$$

H.7. Kalman-filter example

This section provides an example of a Kalman filter with a nonlinear measurement. This highlights the difference between the different types of filters.

H.7.1 Dynamical and measurement model

This example is for a nonlinear spring with a nonlinear measurement. The model is shown in Fig. H.2.

The spring has a cubic spring in parallel with a linear spring. The equations of motion and the measurement equation are

$$\dot{x} = v \tag{H.118}$$
$$m\dot{v} = -cv - k_l x - k_c x^3 + f \tag{H.119}$$
$$\theta = \tan^{-1}\left(\frac{x}{w}\right) \tag{H.120}$$

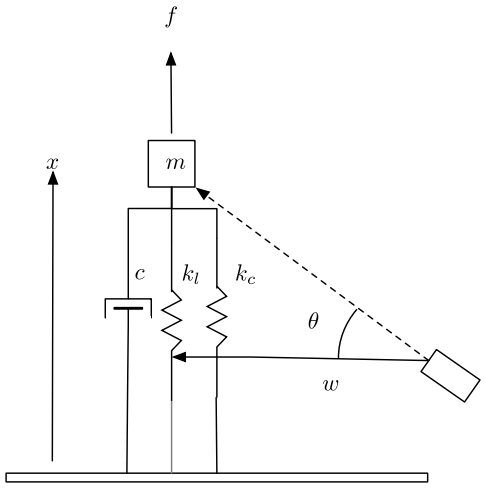


Figure H.2 Mass with a 3rd-order spring in parallel with a damper and linear spring.

The linearized equations, about $x = 0$ are

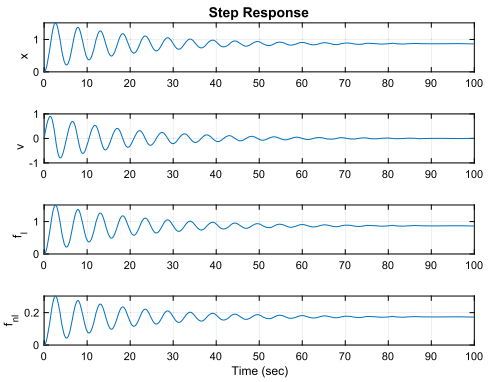
$$\dot{x} = v \tag{H.121}$$

$$m\dot{v} = -cv - k_l x + f \tag{H.122}$$

$$\theta = \frac{x}{w} \tag{H.123}$$

The dynamical model is simulated in Example H.3.

```
d           = RHSNLSpring;
d.f         = 1;
n           = 1000;
dT          = 0.1;
xP          = zeros(4,n);
x           = [0;0];
for k = 1:n
   xP(:,k) = [x;d.kL*x(1);d.kC*x(1)];
   x = RK4(@RHSNLSpring,x,dT,0,d);
end
[t,tL] = TimeLabl((0:n-1)*dT);
Plot2D(t,xP,tL,{'x' 'v' 'f_l' 'f_{nl}'},'Step␣
       Response')
```

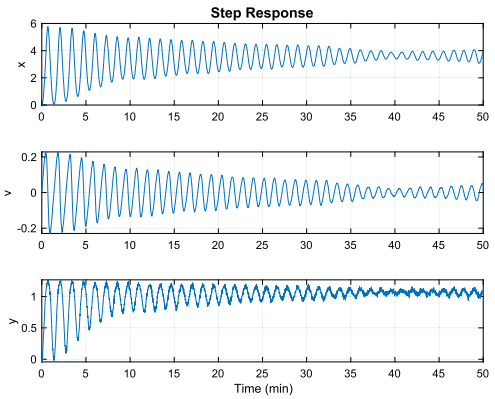


Example H.3: Nonlinear-spring simulation. It shows the nonlinear and linear spring forces.

Example H.4 runs the model once and saves the state and measurement data into a mat file for use in the Kalman-filter demonstrations. The noise obscures the mea-

surement once the position has been damped. The force noise also is visible once the damping has had an effect. The 1σ noise values are in the mat file so that the Kalman filters can set their covariances properly.

```
d             = RHSNLSpring;
f             = 0.01;
d.c           = 2e-3; % Lessen the damping
d.kL          = 1e-4; % Lower the spring constant
d.kC          = 2e-4; % Lower the spring constant
      make it twice the linear spring
noiseForce    = 1e-3; % 1 sigma acceleration noise
noiseMeas     = 0.02; % 1 sigma acceleration noise
n             = 3000; % Number of time steps
dT            = 1;    % Time step, seconds
xP            = zeros(3,n);
x             = [0;0];
w             = 2; % Baseline

for k = 1:n
  y           = atan(x(1)/w) + noiseMeas*randn;
  xP(:,k)     = [x;y];
  d.f         = f + noiseForce*randn;
  x           = RK4(@RHSNLSpring,x,dT,0,d);
end
[t,tL] = TimeLabl((0:n-1)*dT);
Plot2D(t,xP,tL,{'x' 'v' 'y'},'Step Response')

save('KFSim','xP', 'noiseMeas', 'noiseForce', 'd'
       , 'dT', 'n', 'w');
```



Example H.4: Nonlinear-spring simulation generating noisy measurements.

H.7.2 Linear Kalman filter

The first script, KFNLSpring, demonstrates the linear Kalman filter. For the linearized Kalman filter we need the linearized versions of the state and measurement equations around a position x_t. These are found by taking the derivatives of the equations about that point. Given

$$\dot{x} = g(x, u, t) \tag{H.124}$$

$$y = h(x, t) \tag{H.125}$$

The linearized versions at time t are

$$\Phi = \frac{\partial g(x, u, t)}{\partial x}\Big|_t \tag{H.126}$$

$$H = \frac{\partial h(x, t)}{\partial x}\Big|_t \tag{H.127}$$

The derivative of the measurement is

$$\frac{\partial \tan^{-1}\left(\frac{x}{w}\right)}{\partial x} = \frac{w}{w^2 + x^2} \tag{H.128}$$

For $x = 0$

$$H = \begin{bmatrix} \frac{1}{w} & 0 \end{bmatrix} \tag{H.129}$$

the state matrix is

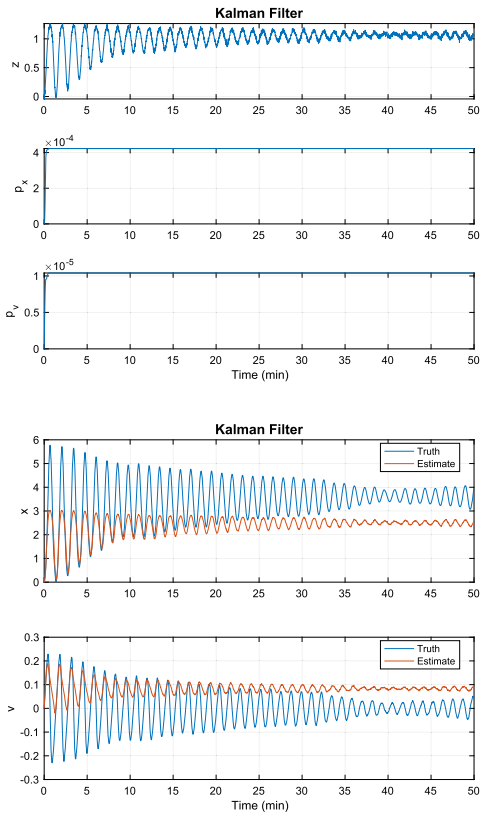
$$a = \begin{bmatrix} 0 & 1 \\ -k_l & -c \end{bmatrix} \tag{H.130}$$

and the input matrix is

$$b = \begin{bmatrix} 0 \\ 1 \end{bmatrix} \tag{H.131}$$

The Kalman-filter code has two parts, measurement update, and state propagation. q is a scalar. e is a 2-by-2 identify matrix. Example H.5 shows the results.

```
s             = load('KFSim');
n             = s.n; % Number of steps
r             = s.noiseMeas^2; % Measurement noise
q             = s.noiseForce^2; % Plant noise
h             = [1/s.w 0]; % Measurement matrix
a             = [0 1;-s.d.kL/s.d.m -s.d.c/s.d.m]; %
        State transition matrix
b             = [0; 1/s.d.m]; % Input matrix

% Discretize
[phi,gamma] = C2DZOH(a,b,s.dT);
xE            = [0;0]; % Estimated state
u             = s.d.f; % Input
p             = [0 0;0 0]; % Initial covariance
xP            = zeros(7,n);
e             = eye(2);

%% Run the Kalman Filter
for j = 1:n

  % Measurement
  z = s.xP(3,j);

  % Store variables to plot
  xP(:,j) = [z;diag(p);s.xP(1:2,j);xE];

  % Kalman Filter measurement update
  k  = p*h'/(h*p*h' + r);
  xE = xE + k*(z - h*xE);
  p  = (e - k*h)*p;

  % Kalman Filter state update
  xE = phi*xE + gamma*u;
  p  = phi*p*phi' + gamma'*q*gamma;

end

%% Plot
yL = {'z' 'p_x' 'p_v' 'x' 'v' };
lL = {{'Truth','Estimate'} {'Truth','Estimate'}};

[t,tL] = TimeLabl(0:(n-1)*s.dT);
Plot2D(t,xP(1:3,:),tL,yL(1:3),'Kalman Filter');
Plot2D(t,xP(4:7,:),tL,yL(4:5),'Kalman Filter',...
  'lin',{'[1 3]' '[2 4]'},[],[],[],[],lL);
```



Example H.5: Conventional (or linear) Kalman filter for a nonlinear spring.

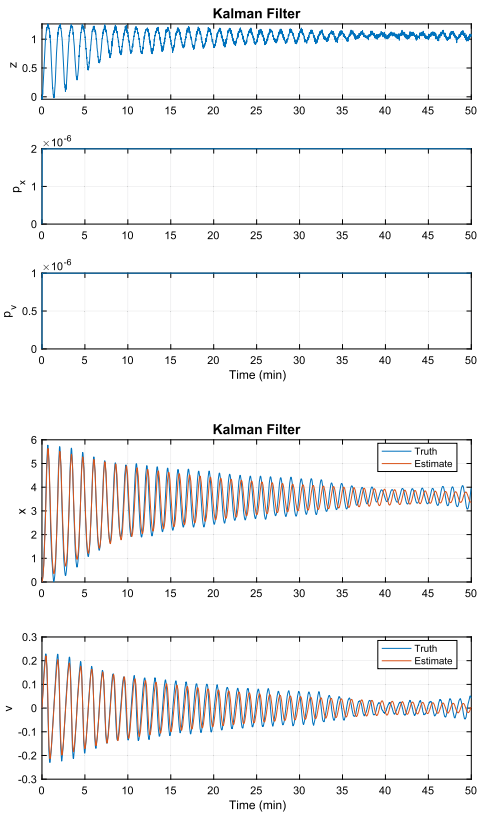
H.7.3 Extended Kalman filter

Example H.6, demonstrates the extended Kalman filter. This requires the functions

1. RHSPartialNLSpring for the partial derivative of the right-hand side;
2. MeasNLSpring for the measurement;
3. MeasPartialNLSpring for the partial derivative of the measurement.

In this case, the partials can be computed analytically. In many cases, these must be done numerically. The results are shown in Example H.6. The estimates track the truth values well. The measurements do come in and out of phase at times.

```
s             = load('KFSim');
n             = s.n; % Number of steps
r             = s.noiseMeas^2; % Measurement noise
q             = s.noiseForce^2; % Plant noise
xE            = [0;0]; % Estimated state
p             = [0 0;0 0]; % Initial covariance
xP            = zeros(7,n);
s.d.w         = s.w;

% Initialize the filter
dEKF          = KFInitialize( 'ekf','f',
        @RHSNLSpring,...
'dT',s.dT,...
'fData',s.d,'h',@MeasNLSpring,...
'hX',@MeasPartialNLSpring,'hData',s.d,...
'fX',@RHSPartialNLSpring,'p',p,...
'q',q,'x',xE,'m',xE,'r',r);

%% EKF Estimation Loop
t = 0;
for k = 1:n
  % Measurement
  z = s.xP(3,k);

  % Store variables to plot
  xP(:,k)         = [z;diag(dEKF.p);s.xP(1:2,k);
          dEKF.m];

        % Extended Kalman Filter
  dEKF.t          = t;
        dEKF.y          = z;
        dEKF            = EKFPredict( dEKF );
        dEKF            = EKFUpdate( dEKF );
  t               = t + s.dT;
end

%% Plot
yL = {'z' 'p_x' 'p_v' 'x' 'v' };
lL = {{'Truth','Estimate'} {'Truth','Estimate'}};

[t,tL] = TimeLabl(0:(n-1)*s.dT);
Plot2D(t,xP(1:3,:),tL,yL(1:3),'Kalman Filter');
Plot2D(t,xP(4:7,:),tL,yL(4:5),'Kalman Filter',...
  'lin',{'[1 3]' '[2 4]'},[],[],[],[],lL);


%-------------------------------------
% $Date$
% $Revision$
```

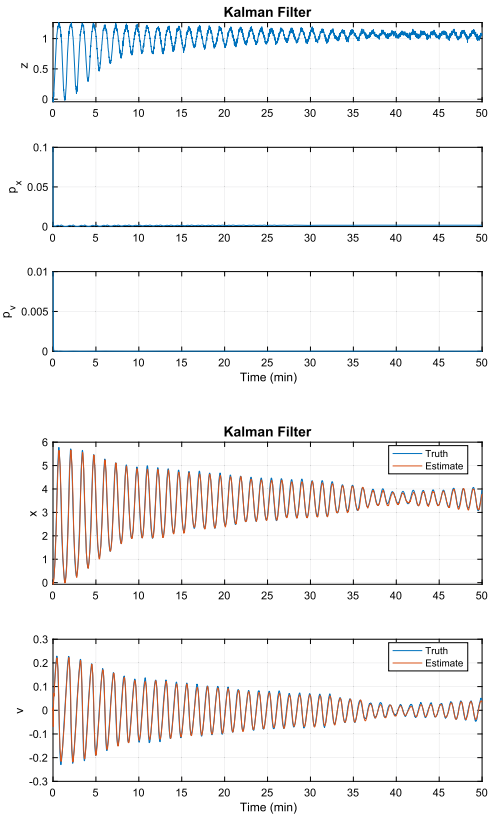


Example H.6: Extended Kalman filter for a nonlinear spring.

H.7.4 Unscented Kalman filter

The script, Example H.7, demonstrates the unscented Kalman filter. It uses the same functions as the simulation. The UKF tracks the pendulum better than the EKF.

```
s             = load('KFSim');
n             = s.n; % Number of steps
r             = s.noiseMeas^2; % Measurement noise
q             = s.noiseForce^2; % Plant noise
xE            = [0;0]; % Estimated state
p             = [0.1 0;0 0.01]; % Initial
        covariance
xP            = zeros(7,n);
s.d.w         = s.w;

% Initialize the filter
dUKF = KFInitialize( 'ukf','f',@RHSNLSpring,...
'dT',s.dT,...
'fData',s.d,'h',@MeasNLSpring,...
'hData',s.d,'p',p,...
'q',q,'x',xE,'m',xE,'r',r);
% Get the UKF weights
dUKF          = UKFWeight( dUKF);

%% UKF Estimation Loop
t = 0;
for k = 1:n
  % Measurement
  z         = s.xP(3,k);

  % Store variables to plot
  xP(:,k) = [z;diag(dUKF.p);s.xP(1:2,k);dUKF.m];

        % Add a measurement source
  dUKF.t                  = t;
        dUKF.y.data             = z;
        dUKF.y.param.hFun           = @MeasNLSpring
                ;
        dUKF.y.param.hData        = s.d;
        dUKF.y.param.r            = r;

  % Unscented Kalman Filter
        dUKF = UKFPredict( dUKF );
        dUKF = UKFUpdate( dUKF );
  t     = t + s.dT;
end

%% Plot
yL = {'z' 'p_x' 'p_v' 'x' 'v' };
lL = {{'Truth','Estimate'} {'Truth','Estimate'}};

[t,tL] = TimeLabl(0:(n-1)*s.dT);
Plot2D(t,xP(1:3,:),tL,yL(1:3),'Kalman Filter');
Plot2D(t,xP(4:7,:),tL,yL(4:5),'Kalman Filter',...
  'lin',{'[1 3]' '[2 4]'},[],[],[],[],lL);
```



Example H.7: Unscented Kalman filter for a nonlinear spring.

References

[1] S. Sarkka, Lecture 3: Bayesian Optimal Filtering Equations and the Kalman Filter, Tech. Rep., Department of Biomedical Engineering and Computational Science, Aalto University School of Science, February 2011.

[2] M.C. VanDyke, J.L. Schwartz, C.D. Hall, Unscented Kalman filtering for spacecraft attitude state and parameter estimation, in: Proceedings of the AAS/AIAA Space Flight Mechanics Conference, 2005, AAS 04-115.

[3] R. van der Merwe, E.A. Wan, The square-root unscented Kalman filter for state and parameter-estimation, in: Proc. IEEE International Conference on Acoustics, Speech and Signal Processing, 2001, pp. 3461–3464.
[4] J. Hartikainen, A. Solin, S. Särkkä, Optimal Filtering with Kalman Filters and Smoothers a Manual for the Matlab toolbox EKF/UKF Version 1.3, Tech. Rep., Aslto University, August 2011.